

Práctica 4: Herramientas de análisis estático de código

Análisis del proyecto *ajedrez* con PMD

Iván Rodríguez Miranda

9 de mayo de 2026

Herramienta	PMD 7.7.0 (Maven PMD Plugin 3.26.0)
Proyecto evaluado	ajedrez (Java 21, Maven)
Fecha del análisis	09/05/2026
Archivos analizados	21 ficheros . java
Líneas de código	1145 líneas en src/main/java

Índice

1. Objetivo de la práctica	2
2. Descripción del código Java evaluado	2
2.1. Distribución por paquetes	2
2.2. Trabajo de código realizado antes del análisis	3
3. Instalación y configuración de PMD	3
4. Reglas utilizadas y justificación	4
4.1. Categorías seleccionadas y justificación	5
5. Secuencia de comandos ejecutados	5
6. Resultados obtenidos con PMD	6
6.1. Ficheros generados	6
6.2. Resumen numérico de las violaciones	6
6.3. Limitación observada del entorno	6
7. Listado e interpretación de los errores detectados	7
7.1. Tipo 1: ConstructorCallsOverridableMethod (Error Prone, prioridad 1) .	7
7.2. Tipo 2: CompareObjectsWithEquals (Error Prone, prioridad 3)	7

7.3. Tipo 3:NullAssignment (Error Prone, prioridad 3)	8
7.4. Tipo 4:LocalVariableCouldBeFinal (Code Style, prioridad 3)	8
7.5. Tipo 5:UnnecessaryConstructor (Code Style, prioridad 3)	8
7.6. Tipo 6:AvoidInstantiatingObjectsInLoops (Performance, prioridad 3) .	9
8. Tabla resumen de defectos por característica ISO/IEC 25010	10
9. Relación con la calidad del software según ISO/IEC 25000	10
10. Conclusiones	11
Anexo A: Evidencias de salida	11
Anexo B: Materiales y bibliografía	12

1 Objetivo de la práctica

El objetivo de esta práctica es aplicar una herramienta de análisis estático de código sobre un proyecto Java propio, interpretar los resultados obtenidos y relacionarlos con las características de calidad del software definidas por la familia de estándares ISO/IEC 25000 (modelo de calidad SQuaRE). Para ello se utiliza **PMD**, un analizador estático que aplica conjuntos de reglas para detectar posibles defectos, código no utilizado, patrones propensos a error, malas prácticas de programación y problemas de mantenibilidad o rendimiento.

2 Descripción del código Java evaluado

El código evaluado corresponde a una aplicación Java de ajedrez organizada en paquetes de modelo, vista y controlador siguiendo el patrón MVC. La parte más desarrollada es el modelo (tablero, casillas y piezas) junto con el controlador del juego.

Dato	Valor
Fecha del análisis	09/05/2026
Lenguaje	Java
Versión de compilación (release)	Java 21
JVM usada en la ejecución	OpenJDK 25.0.2
Sistema de construcción	Apache Maven 3.8.5
Archivos . java evaluados	21
Líneas totales en src/main/java	1145

2.1 Distribución por paquetes

Paquete	Responsabilidad
com.ajedrez	Clase principal App (punto de entrada JavaFX).
com.ajedrez.modelo	Tablero, casillas, color y clase abstracta Pieza.
com.ajedrez.modelo.piezas	Reglas de movimiento de Alfil, Caballo, Peón, Rey, Reina y Torre.
com.ajedrez.controlador	ControladorJuego y ControladorTablero.
com.ajedrez.vista	VistaTablero (interfaz JavaFX).

2.2 Trabajo de código realizado antes del análisis

Archivo	Actuación
Torre.java	Implementada como subclase de Pieza con validación de movimientos rectos, control de piezas intermedias y captura.
Reina.java	Combina movimientos rectos y diagonales con camino libre y captura válida.
Rey.java	Permite un único desplazamiento de una casilla en cualquier dirección.
Caballo.java	Validación del movimiento en L y documentación.
Peon.java	Avance según color, doble avance inicial, bloqueo frontal y captura diagonal. No contempla captura al paso.
package-info.java	Documentación Javadoc de cada paquete.
App.java	Punto de entrada JavaFX y configuración de la ventana principal.
VistaTablero.java	Tablero 8x8 con piezas, selección, turnos, mensajes y reinicio.
ControladorJuego.java	Coordinación de tablero, turno actual, selección y movimiento.
Tablero.java	Inicialización de piezas, limpieza, colocación y movimiento validado.
ajedrez.css	Hoja de estilos JavaFX para el tablero y los estados visuales.

3 Instalación y configuración de PMD

PMD se ha integrado a través de Maven utilizando el plugin oficial `maven-pmd-plugin`, definido en `pom.xml` tanto en `<build>` (para ejecutar el goal `check` en cada `verify`) como en `<reporting>` (para generar el informe HTML mediante `site`).

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-pmd-plugin</artifactId>
  <version>3.26.0</version>
  <configuration>
    <targetJdk>21</targetJdk>
    <failOnViolation>>false</failOnViolation>
    <printFailingErrors>>true</printFailingErrors>
    <linkXRef>>false</linkXRef>
    <rulesets>
      <ruleset>${project.basedir}/pmd-ruleset.xml</ruleset>
    </rulesets>
    <excludes>
      <exclude>com/ajedrez/App.java</exclude>
      <exclude>com/ajedrez/vista/**</exclude>
    </excludes>
  </configuration>
  <executions>
    <execution>
      <goals>
        <goal>check</goal>
      </goals>
    </execution>
  </executions>
```

```
</plugin>
```

Listing 1: Configuración de PMD en pom.xml

Decisiones de configuración:

- `targetJdk=21` fija el nivel de compatibilidad de análisis con la versión de compilación declarada en el proyecto.
- `failOnViolation=false` permite obtener el informe completo sin que el ciclo Maven se interrumpa cuando aparecen violaciones, algo deseable para una práctica académica donde el interés es analizar los hallazgos, no bloquear la build.
- Se excluyen `App.java` y todo el paquete `com/ajedrez/vista/**`: en el entorno usado (JDK 25.0.2) PMD lanza errores internos al resolver tipos JavaFX. La exclusión mantiene el análisis estable sobre el modelo y los controladores, que son la parte central de la lógica del juego.
- Se utiliza un **ruleset propio**, más completo que el ruleset por defecto del plugin Maven, para poder evaluar características de calidad con suficiente cobertura.

4 Reglas utilizadas y justificación

Se ha definido un ruleset propio en `pmd-ruleset.xml` que combina seis categorías oficiales de PMD. La elección busca cubrir las dimensiones más relevantes de la calidad del código en un proyecto en fase inicial (correctitud, claridad, mantenibilidad, rendimiento) sin caer en reglas excesivamente estrictas que generan ruido en código académico (longitud de nombres, número de retornos, etc.).

```
<ruleset name="Ajedrez PMD Ruleset"
  xmlns="http://pmd.sourceforge.net/ruleset/2.0.0">
  <rule ref="category/java/bestpractices.xml"/>
  <rule ref="category/java/codestyle.xml">
    <exclude name="OnlyOneReturn"/>
    <exclude name="LongVariable"/>
    <exclude name="ShortVariable"/>
    ...
  </rule>
  <rule ref="category/java/design.xml">
    <exclude name="LawOfDemeter"/>
    <exclude name="LoosePackageCoupling"/>
    <exclude name="DataClass"/>
  </rule>
  <rule ref="category/java/errorprone.xml"/>
  <rule ref="category/java/multithreading.xml"/>
  <rule ref="category/java/performance.xml"/>
</ruleset>
```

Listing 2: Extracto del ruleset propio `pmd-ruleset.xml`

4.1 Categorías seleccionadas y justificación

Categoría	Justificación de uso	Características ISO/IEC 25000
bestpractices	Detecta código no utilizado, malos hábitos generales y construcciones que dificultan la evolución.	Mantenibilidad, fiabilidad.
codestyle	Garantiza claridad y legibilidad uniforme. Reduce el ruido sintáctico que esconde defectos.	Mantenibilidad.
design	Identifica clases o métodos con diseño mejorable: complejidad accidental, acoplamiento, abstracciones débiles.	Mantenibilidad, adecuación funcional.
errorprone	Avisa de patrones con alta probabilidad de producir errores en ejecución. Es la categoría más crítica.	Fiabilidad, seguridad.
multithreading	Aunque el proyecto aún no usa hilos, se mantiene como red de seguridad ante futuras extensiones (motor de IA, temporizadores).	Fiabilidad, eficiencia de desempeño.
performance	Detecta patrones ineficientes (creación innecesaria de objetos, bucles, etc.).	Eficiencia de desempeño.

5 Secuencia de comandos ejecutados

Todos los comandos se ejecutan desde la raíz del proyecto.

```
# Inventario y caracterizacion del codigo
find src/main/java -name '*.java' -print
wc -l $(find src/main/java -name '*.java')
java -version
./mvnw -version

# Ciclo de Maven y analisis con PMD
./mvnw test
./mvnw verify
./mvnw pmd:pmd          # genera target/pmd.xml
./mvnw pmd:check        # falla la build si failOnViolation=true
./mvnw site             # genera el informe HTML target/reports/pmd.html

# Ejecucion de la aplicacion JavaFX (verificacion funcional)
./mvnw javafx:run
```

Listing 3: Comandos de análisis e inventariado del código

Comando	Resultado
find src/main/java -name '*.java'	Localiza 21 ficheros Java.
wc -l ...	1145 líneas Java en total.
java -version	OpenJDK 25.0.2.
./mvnw -version	Apache Maven 3.8.5 sobre Java 25.0.2.
./mvnw test	BUILD SUCCESS; sin tests definidos aún.
./mvnw verify	BUILD SUCCESS; informe PMD generado.
./mvnw pmd:pmd	Genera target/pmd.xml.
./mvnw site	Genera target/reports/pmd.html navegable.
./mvnw javafx:run	Lanza la ventana JavaFX (verificación visual).

6 Resultados obtenidos con PMD

6.1 Ficheros generados

Fichero	Contenido
target/pmd.xml	Informe completo en formato XML.
target/reports/pmd.html	Informe HTML navegable generado por site.
pmd-ruleset.xml	Ruleset propio aplicado en el análisis.

6.2 Resumen numérico de las violaciones

PMD ha detectado un total de **46 violaciones** sobre los 14 ficheros Java incluidos en el alcance del análisis (modelo + controlador, excluida la vista JavaFX).

Categoría (ruleset)	Número de violaciones
Code Style	25
Error Prone	20
Performance	1
Total	46

Regla	Categoría	Ocurrencias
LocalVariableCouldBeFinal	Code Style	23
CompareObjectsWithEquals	Error Prone	15
NullAssignment	Error Prone	4
UnnecessaryConstructor	Code Style	2
ConstructorCallsOverridableMethod	Error Prone	1
AvoidInstantiatingObjectsInLoops	Performance	1

6.3 Limitación observada del entorno

Durante la ejecución, PMD muestra trazas internas con el mensaje `Unsupported class file major version 69` cuando intenta resolver clases compiladas con OpenJDK 25 (versión mayor 69). El proyecto se compila con Java 21, pero la JVM que ejecuta Maven es la 25, y PMD/ASM 9 no reconoce todavía ese nivel de bytecode. Esta incidencia debe documentarse como una **limitación del**

entorno de análisis, no como un defecto del código: el informe de PMD se genera correctamente y todas las reglas evalúan el código fuente.

7 Listado e interpretación de los errores detectados

A continuación se analiza un **ejemplo representativo de cada tipo** de violación detectada (las repetidas no se duplican). Para cada caso se describe el patrón, su nivel de importancia para la calidad del software y la característica del modelo ISO/IEC 25010 con la que se relaciona principalmente.

7.1 Tipo 1: ConstructorCallsOverridableMethod (Error Prone, prioridad 1)

Localización: ControladorJuego.java, línea 27.

Mensaje PMD: *Overridable method called during object construction: ControladorJuego.reiniciarPartida().*

Código afectado:

```
public ControladorJuego() {  
    this.tablero = new Tablero();  
    reiniciarPartida();    // <-- metodo no final, llamado desde el  
                           constructor  
}
```

Descripción: llamar dentro del constructor a un método que puede ser sobrescrito por una subclase es peligroso, porque la subclase aún no se ha inicializado cuando el método se ejecuta. Esto puede dar lugar a estados parcialmente inicializados.

Importancia: alta (prioridad 1 en PMD). Es un patrón clásico de defectos sutiles difíciles de depurar.

Característica ISO/IEC 25010 relacionada: *Fiabilidad (maturity/fault tolerance)*: el patrón aumenta la probabilidad de fallos en ejecución cuando se extiende la clase. También afecta a la *Mantenibilidad (modifiability)* porque limita la libertad de extender la clase sin riesgo.

7.2 Tipo 2: CompareObjectsWithEquals (Error Prone, prioridad 3)

Localización (ejemplo): Peon.java, línea 83 (y 14 ocurrencias más).

Mensaje PMD: *Use equals() to compare object references.*

Código afectado:

```
return avanceFila == obtenerDireccion()  
    && avanceColumna == 1  
    && !destino.estaVacía()  
    && destino.getPiezaActual().getColor() != getColor();
```

Descripción: se comparan referencias ColorPieza con != en lugar de != equals(...). En este caso concreto ColorPieza es un enum y la comparación con == es correcta y deseable, pero PMD no puede inferirlo en todos los casos y por seguridad recomienda equals. La regla obliga a justificar explícitamente cada uso de == sobre objetos.

Importancia: media. Aunque para enums el patrón es seguro, en otros tipos provocaría errores funcionales (comparar dos Casilla distintas pero equivalentes, por ejemplo).

Característica ISO/IEC 25010 relacionada: *Fiabilidad* (comparaciones erróneas conducen a comportamiento incorrecto) y *Mantenibilidad (analyzability)*: el código se vuelve frágil ante refactorizaciones que sustituyan un enum por una clase normal.

7.3 Tipo 3: NullAssignment (Error Prone, prioridad 3)

Localización (ejemplo): ControladorJuego.java, línea 63.

Mensaje PMD: *Assigning an Object to null is a code smell. Consider refactoring.*

Código afectado:

```
public void reiniciarPartida() {  
    tablero.inicializarPiezas();  
    turnoActual = ColorPieza.BLANCO;  
    casillaSeleccionada = null;    // <-- asignacion a null  
}
```

Descripción: asignar null a un campo se considera una señal de modelado deficiente: idealmente el estado de “no hay selección” se representa con un objeto *Optional* o con un patrón *Null Object*.

Importancia: baja-media. Aquí el uso es legible y está controlado; pero introduce null en el resto del flujo y obliga a comprobaciones repartidas.

Característica ISO/IEC 25010 relacionada: *Mantenibilidad (analyzability y modifiability)* y *Fiabilidad (fault tolerance)*: el uso intensivo de null aumenta el riesgo de NullPointerException en futuras extensiones.

7.4 Tipo 4: LocalVariableCouldBeFinal (Code Style, prioridad 3)

Localización (ejemplo): Pieza.java, líneas 42–48 (y 22 ocurrencias más).

Mensaje PMD: *Local variable 'xxx' could be declared final.*

Código afectado:

```
int avanceFila = Integer.compare(destino.getFila(), origen.getFila());  
int avanceColumna = Integer.compare(destino.getColumna(),  
    origen.getColumna());  
int filaActual = origen.getFila() + avanceFila;  
int columnaActual = origen.getColumna() + avanceColumna;
```

Descripción: las dos primeras variables nunca se reasignan; podrían declararse final. Marcar como final las variables locales documenta la inmutabilidad y previene reasignaciones accidentales durante futuras modificaciones.

Importancia: baja. No es un defecto funcional, pero mejora la legibilidad y evita errores en evoluciones del código.

Característica ISO/IEC 25010 relacionada: *Mantenibilidad (analyzability, modifiability)*: final hace explícito el contrato de cada variable y reduce la carga cognitiva al leer el método.

7.5 Tipo 5: UnnecessaryConstructor (Code Style, prioridad 3)

Localización (ejemplo): ControladorTablero.java, línea 15.

Mensaje PMD: *Avoid unnecessary constructors — the compiler will generate these for you.*

Código afectado:

```
public class ControladorTablero {  
    public ControladorTablero() {  
        // constructor vacio que duplica al sintetizado por el compilador  
    }  
}
```

Descripción: declarar un constructor vacío público sin contenido es redundante: Java genera automáticamente uno equivalente. El código añadido no aporta semántica.

Importancia: **baja.** Es ruido de código (*code noise*).

Característica ISO/IEC 25010 relacionada: *Mantenibilidad (analyzability)*: cuanto más breve y limpio sea el código, más rápido se entiende.

7.6 Tipo 6: AvoidInstantiatingObjectsInLoops (Performance, prioridad 3)

Localización: Tablero.java, línea 116.

Mensaje PMD: *Avoid instantiating new objects inside loops.*

Código afectado:

```
private void inicializarPeones(final ColorPieza color, final int fila) {  
    for (int columna = 0; columna < TAMANO_TABLERO; columna++) {  
        colocarPieza(fila, columna, new Peon(color)); // <-- new dentro  
        del bucle  
    }  
}
```

Descripción: se crea un objeto nuevo en cada iteración. En este caso es inevitable (cada casilla necesita su propia instancia de Peon), pero la regla sigue siendo útil en código donde el objeto pudiera reutilizarse.

Importancia: **baja** en este contexto (se ejecuta una sola vez por partida). Sería **alta** en un motor de búsqueda de movimientos donde se genera el árbol de jugadas.

Característica ISO/IEC 25010 relacionada: *Eficiencia de desempeño (resource utilization, time behaviour)*: la creación excesiva de objetos presiona al recolector de basura y degrada el tiempo de respuesta cuando el número de iteraciones crece.

8 Tabla resumen de defectos por característica ISO/IEC 25010

Regla detectada	Característica ISO/IEC 25010 dominante	Severidad
ConstructorCallsOverridableMethod	Fiabilidad / Mantenibilidad	Alta
CompareObjectsWithEquals	Fiabilidad / Mantenibilidad	Media
NullAssignment	Mantenibilidad / Fiabilidad	Baja-Media
LocalVariableCouldBeFinal	Mantenibilidad	Baja
UnnecessaryConstructor	Mantenibilidad	Baja
AvoidInstantiatingObjectsInLoops	Eficiencia de desempeño	Baja

9 Relación con la calidad del software según ISO/IEC 25000

La familia ISO/IEC 25000 (SQuaRE) propone evaluar la calidad del producto software mediante ocho características: *adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad*. PMD aporta evidencia sobre todo en cuatro de ellas:

Característica	Aporte de PMD	Aplicación al proyecto ajedrez
Mantenibilidad	Detecta código muerto, constructores innecesarios, variables que pueden ser final, estructuras duplicadas y patrones que dificultan la lectura.	Mejora la facilidad para revisar las clases de piezas y los controladores y para extenderlos a nuevas reglas (jaque, enroque, captura al paso).
Fiabilidad	Identifica patrones propensos a fallos: comparaciones con == sobre objetos, NullPointerException potenciales, métodos sobrescribibles desde el constructor.	Reduce la probabilidad de movimientos inválidos, estados incoherentes del tablero y errores en el cambio de turno.
Eficiencia de desempeño	Reglas como AvoidInstantiatingObjectsInLoops o las de tipo Big*Instantiation avisan sobre patrones costosos.	Hoy el impacto es bajo, pero será importante si se añade IA, búsqueda de jugadas o validación masiva de partidas.
Seguridad	Reglas como AvoidUsingHardCodedIP o HardCodedCryptoKey bloquean malas prácticas.	Hoy el proyecto no maneja red ni credenciales, pero la cobertura preventiva sigue activa.

Limitaciones del análisis estático: ningún analizador estático puede sustituir las pruebas funcionales. PMD no garantiza que la torre se mueva en línea recta ni que el peón capture en diagonal: solo asegura que el código no presenta los patrones recogidos en sus reglas. La adecuación funcional debe verificarse con pruebas unitarias (JUnit) que se proponen como próxima mejora.

10 Conclusiones

PMD se ha integrado correctamente en el ciclo Maven del proyecto y se ha aplicado un ruleset propio que incluye seis categorías oficiales (bestpractices, codestyle, design, errorprone, multithreading y performance). El análisis del 09/05/2026 sobre 21 archivos Java y 1145 líneas ha generado los siguientes hallazgos principales:

- Se han detectado **46 violaciones** agrupadas en seis tipos de regla.
- El único defecto de prioridad máxima (1) es la llamada al método sobrescribible `reiniciarPartida()` desde el constructor de `ControladorJuego`; conviene marcar `ControladorJuego` como `final` o el método como `final/private` para evitarlo.
- La mayoría de hallazgos (23 `LocalVariableCouldBeFinal` y 15 `CompareObjectsWithEquals`) son de baja severidad pero refuerzan la mantenibilidad y la prevención de errores futuros si se aplican.
- El uso de `NullAssignment` sugiere migrar a `Optional<Casilla>` para representar la “casilla seleccionada”.
- La regla de *performance* en `Tablero.inicializarPeones` es tolerable porque se ejecuta una sola vez al iniciar la partida.

Próximas mejoras sugeridas:

1. Añadir tests JUnit para cada pieza (Torre, Alfil, Caballo, Peón, Reina, Rey) que verifiquen movimientos legales e ilegales y la captura.
2. Marcar `ControladorJuego` como `final` y `reiniciarPartida` como `private`, eliminando la violación de prioridad 1.
3. Sustituir las asignaciones `= null` por `Optional<Casilla>`.
4. Habilitar `failOnViolation=true` cuando el código esté libre de hallazgos críticos para integrar PMD como puerta de calidad en CI.

Anexo A: Evidencias de salida

```
[INFO] BUILD SUCCESS
[INFO] PMD Version: 7.7.0
[INFO] Total time: ...
```

Listing 4: Salida resumida de `./mvnw verify`

```
<pmd version="7.7.0" timestamp="2026-05-09T14:23:51.910">
  <file name="../../../ControladorJuego.java">
    <violation beginline="27" rule="ConstructorCallsOverridableMethod"
      ruleset="Error Prone" priority="1">
      Overridable method called during object construction:
      ControladorJuego.reiniciarPartida()
    </violation>
    <violation beginline="63" rule="NullAssignment"
      ruleset="Error Prone" priority="3">
      Assigning an Object to null is a code smell. Consider refactoring.
    </violation>
    ...
```

```
</file>
...
</pmd>
```

Listing 5: Extracto del informe `target/pmd.xml`

```
[WARNING] Unsupported class file major version 69
```

Listing 6: Limitación del entorno (no es un defecto del código)

Anexo B: Materiales y bibliografía

- PMD — documentación oficial: <https://pmd.github.io/>
- Maven PMD Plugin — página del proyecto: <https://maven.apache.org/plugins/maven-pmd-plugin/>
- ISO/IEC 25000 SQuaRE: <https://iso25000.com/>
- Javier Garzás — herramientas de calidad: <https://www.javiergarzas.com/>
- BairesDev — Java static code analysis tools: <https://www.bairesdev.com/blog/java-static-code-a>